Alberi Rosso-nero

Materia	Algoritmi
Anno	Secondo Anno
Data	@November 14, 2024
Studiato	Done

Definizione

Un **albero rosso-nero** o **RB-Tree** è un'**estensione di un BST**, ossia di un albero binario di ricerca. In particolare, sono dei BST **auto-bilanciati**.
L'albero binario di ricerca è una struttura dati particolarmente potente poiché tutte le operazioni vengono eseguite con un tempo proporzionale all'altezza dell'albero. Nel caso pessimo, l'altezza massima dell'albero è n se n è il numero di elementi, altrimenti è $O(\log n)$. Con gli alberi standard, non abbiamo questa garanzia.

Possiamo modificare l'albero binario di ricerca introducendo una piccola "intelligenza", ossia permettendo che questo si **auto-bilanci**, verificando se ad ogni operazione si perde o meno la proprietà di bilanciamento. Quindi il vantaggio è una struttura che **funzioni come un BST ma ha sempre altezza** $h \in O(\log n)$.

Nell'albero binario di ricerca vale la seguente proprietà:

> *Per ogni nodo, **tutti i nodi** del sottoalbero sinistro devono avere chiave più piccola di quella del nodo considerato, e tutti i nodi del sottoalbero destro devono avere chiave maggiore o uguale di quella del nodo considerato.*

L'albero rosso-nero aggiunge una cosa in più. I nodi hanno un campo aggiuntivo: il **campo colore**. Il campo è un campo a 1 bit e il colore è o rosso o nero.

Regole

Vi sono 5 regole da seguire per la rappresentazione di un RB-Tree:

1. ogni nodo è o rosso o nero;
2. la radice **è nera**;
3. le foglie **sono nere**. Applichiamo un **rilassamento** di tale proprietà: ignoriamo tale regola e **supporremo che tale proprietà sia sempre valida**. Questo perché le foglie sono nodi i cui figli sono $nihil$. Diciamo quindi che **le foglie sono i** $nihil$. Se tali nodi sono di default neri, allora la proprietà è rispettata. Aggiungendo questi nodi, allora **raddoppiamo il numero di nodi**, passando da $n \to 2n$. La complessità algoritmica quindi **non varia**. Inoltre, posso semplicemente aggiungere **un unico nodo** $nihil$ **nero a cui si collegano tutte le foglie.** Tale considerazione quindi è richiesta solamente nella implementazione, per cui nelle nostre astrazioni non rappresenteremo tale nodo;
4. **un nodo rosso ha solo figli neri** (=un nodo rosso **è figlio solo di un nodo nero**, un nodo rosso **non ha figli rossi**);
5. **un cammino da un qualsiasi nodo ad una sua foglia (discendente) qualsiasi contiene lo stesso numero di nodi neri**. Quindi partendo da un nodo x ad una foglia, si incontra un numero di nodi m neri. Il numero di nodi neri che si incontrano da un nodo ad una foglia x' è **sempre** m. Tale numero prende il nome di ***black height*** $bh(x) = m$ **o altezza nera di** x. Tale altezza vale **per ogni foglia**.

Albero con soli nodi neri

> *Un albero con tutti i nodi neri dev'essere* ***perfettamente bilanciato****, ossia* ***completo****.*

Fattore di sbilanciamento

> *Il ramo più lungo non potrà mai essere uguale a più del doppio del ramo più corto.*

Ne segue che:

$$h \leq 2\log(n+1)$$

Bilanciamento

Non basta tuttavia definire tali proprietà per garantire il bilanciamento. Nel momento in cui dobbiamo inserire un nodo, dobbiamo inserire delle procedure che abbiamo un tempo non superiore rispetto all'inserimento e che permettano il mantenimento le proprietà viste.

Rotazioni, rotazioni destre e sinistre

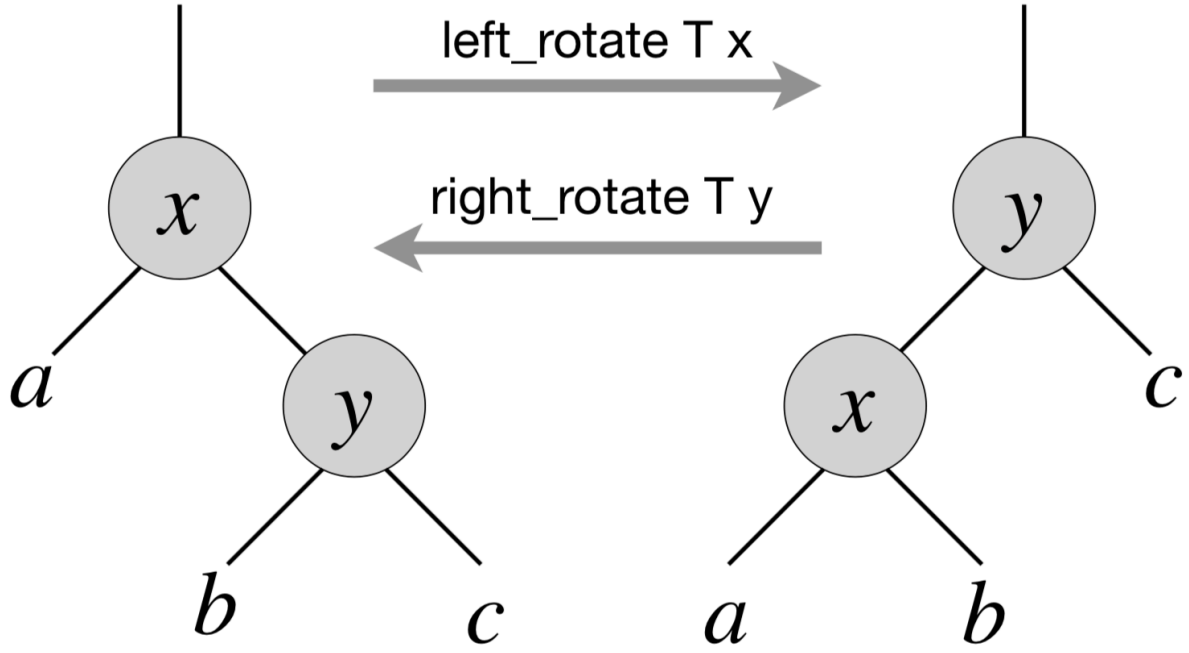
Le operazioni eseguite sull'albero rosso-nero potrebbero violare le proprietà viste. Pertanto, nel caso di violazioni dovremmo mutare:

- il colore dei nodi;
- la struttura dei puntatori.

Analizzeremo per prima quest'ultima, introducendo le **rotazioni**.

> *Una **rotazione** è un'operazione locale in un albero di ricerca che preserva la proprietà degli alberi binari di ricerca.*

Consideriamo la seguente situazione:



Il nodo x ha un figlio destro y e un sottoalbero sinistro a (tale sottoalbero, per definizione ricorsiva dell'albero di ricerca, può essere vuoto. Si sottolinea pertanto che y **non dev'essere nullo**). La rotazione sinistra modifica tale sottoalbero radicato su x "facendo perno" sul collegamento tra i nodi x e y. La radice di tale sottoalbero diventa y, che ha come figlio sinistro il nodo x e come figlio destro il suo sottoalbero originario destro c. Il suo sottoalbero sinistro b, invece, diventa figlio destro del nodo x.
Nella rotazione destra, si fa sempre perno sul collegamento tra x e y. Allora y diventa figlio destro di x, e il sottoalbero figlio destro di x diventa figlio sinistro di y. x diventa la nuova radice con figlio sinistro invariato a.

Le rotazioni richiedono un tempo costante.

Procedure dell'albero rosso-nero

Inserimento

L'operazione di inserimento ristabilisce dopo l'inserimento del nuovo nodo le proprietà dell'albero rosso-nero.

Ogni volta che viene inserito un nuovo nodo, esso verrà inserito al livello delle foglie. Quando viene creato un nuovo nodo, per la proprietà $1.$ tale nodo deve assumere un colore:

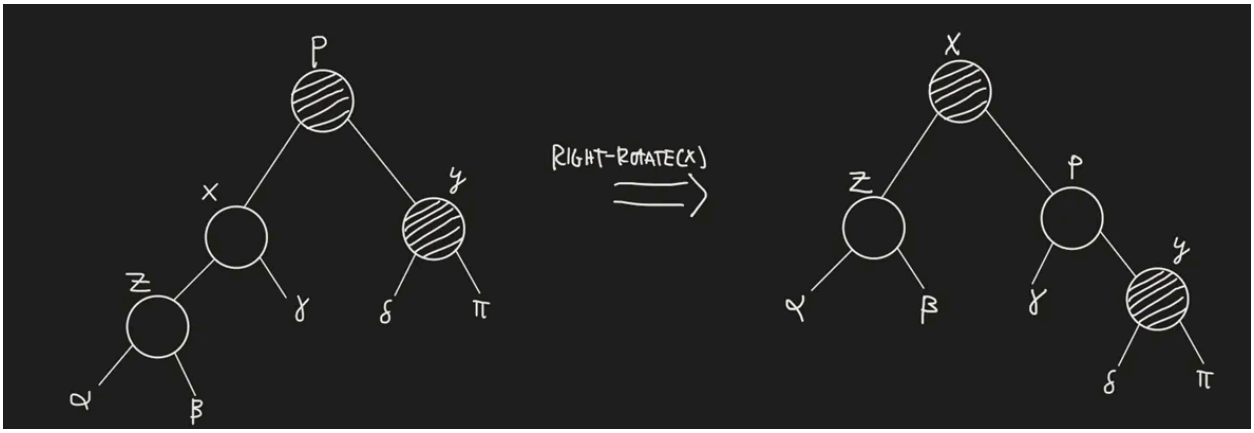
- se lo coloriamo di nero, possiamo rispettare la proprietà $4.$ tranquillamente, poiché un nodo rosso ha soli figli neri. Tuttavia, **violiamo la proprietà** 5, poiché alcuni percorsi si trovano un nodo nero in più;
- se lo coloriamo di rosso, rispettiamo sicuramente la proprietà $5.$ perché non alteriamo l'altezza nera. Tuttavia, **violiamo la proprietà** $4.$, poiché se lo inseriamo come figlio di un nodo rosso diventa un problema. Ma **questo non crea problemi se inseriamo il nodo rosso come figlio di un nodo nero**.

Risulta più vantaggioso ristabilire la proprietà 4. Quindi **inseriremo con colore rosso**.

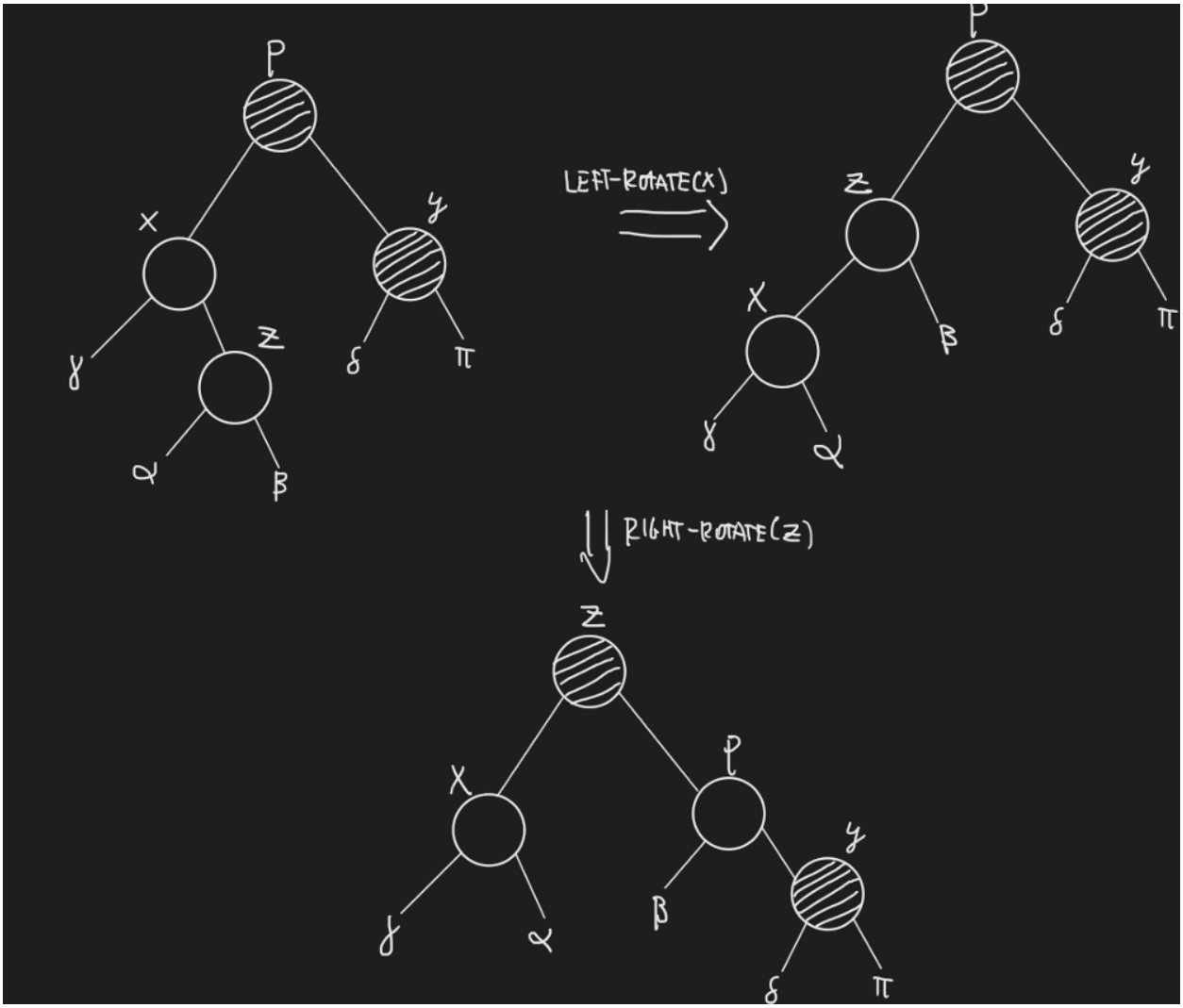
RB-Insert-Fixup: Procedura per eliminazione della violazione della proprietà $4.$

Ci sono solo 3 casi da considerare. Supponiamo di avere un nodo x rosso con padre P nero. Supponiamo P abbia anche figlio y. Supponiamo di voler inserire un nodo z figlio di x:

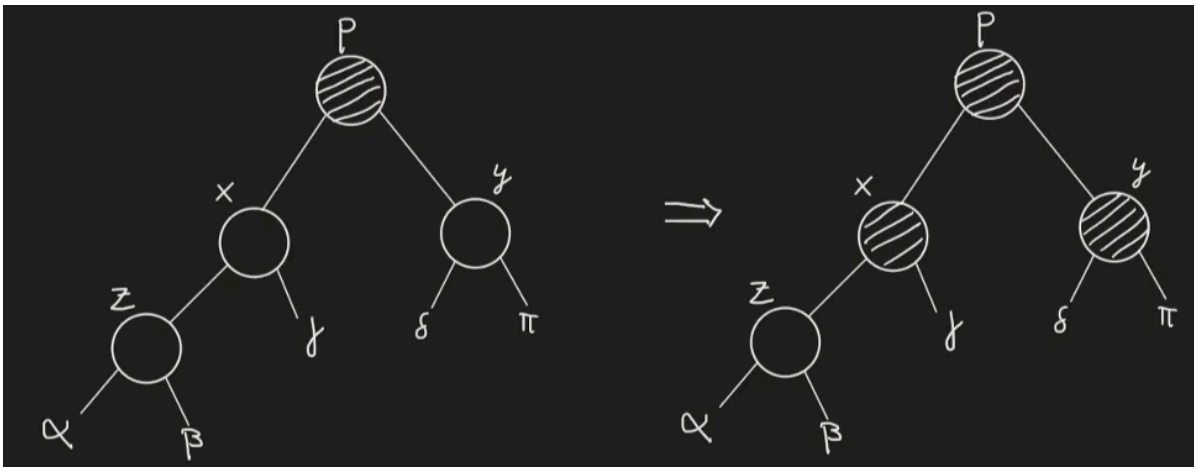
1. il primo caso è se y **è nero e** z **è figlio esterno di** x. Allora in questo caso ruotiamo x così che occupi il posto del parent P, cambiamo il colore di x rendendolo **nero** e rendiamo P rosso. Notiamo che se z, y sono radici di due sottoalberi per cui vale la proprietà $5.$, tale proprietà **viene rispettata comunque:**



2. il secondo caso è se y **è nero e il nodo da inserire** z **è figlio interno di** x. In questo caso si ruota su x per ritornare al caso 1;



3. il terzo caso è se y **è rosso**. Se provassimo ad agire con le rotazioni, otterremmo sempre due nodi consecutivi rossi. In questo caso **è necessario propagare il nodo nero ai figli**, ossia P diventa **rosso** e x, y diventano **neri**. Chiaramente se P è radice, esso va posto nuovamente come nodo nero.
A seguito di tale operazione, comunque, potremmo aver violato la proprietà 5. in base al colore del predecessore di P. Allora si richiama la procedura per il nodo P.



Caso in cui P è radice: i figli diventano neri, e poiché P è radice per la proprietà 2 esso dev'essere nero.